Day 118 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 17, Day 7 reference

Watch: Nested Cross Validation - Algorithm Explained (Machine Learning Mastery) | Time Series Cross Validation Explained (The NextGen Ai Frontier)

`day118_model_selection.py`, segment by segment, in the order `main()` uses things. The six exercise parts (Days 112-117) are copied verbatim and were explained on their own days; this guide recaps them in one segment, then walks the new code: the fold makers, the models built from the parts, the time-series helpers and the experiments. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: k = number of folds (or of neighbours, for kNN) · tr, va, te = training, validation and test rows · lags = past values used as features

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.join(HERE, "..", "week-15-practical-nlp-pretrained"))
with contextlib.redirect_stdout(io.StringIO()):
    import day99_exercise as d99
EPS = 1e-6
REPEATS = 50
DRAWS = 40
SPLITS = 10
LAGS = 8
KS = [1, 3, 5, 10, 20, 40]
SKIP = {(115, 'sqdist'), (117, 'sqdist')}
HELPERS = {'spy', 'useful', 'useless'}
```

- `EPS = 1e-6` — Day 116's part uses a module-level constant, so the build must define it too.
- `SKIP = {(115, 'sqdist'), (117, 'sqdist')}` — a set of (day, name) pairs: copies of Day 113's sqdist left out.
- `HELPERS = {'spy', 'useful', 'useless'}` — Day 116's hand-check helpers: tools for its own check, not parts.

Segment 1 — The week's parts

The week's parts, verbatim (one banner per exercise file)

CODE

```
# from day112_exercise.py, verbatim: smo, bias, decision
# ============================================================================
def smo(K, y, C, tol=1e-3, max_iter=200000):            # NEW: Day 112's solver, unchanged
...  (Days 113 to 117 follow, each under its own banner)
```

- `# from dayNNN_exercise.py, verbatim: ...` — a generator pasted the parts in and wrote these banners; nothing was retyped.
- `smo, rbf, knn_predict, scores` — the four parts the experiments actually call: the SVM solver, the RBF kernel, kNN's vote and Day 117's metrics.

Segment 2 — The fold makers

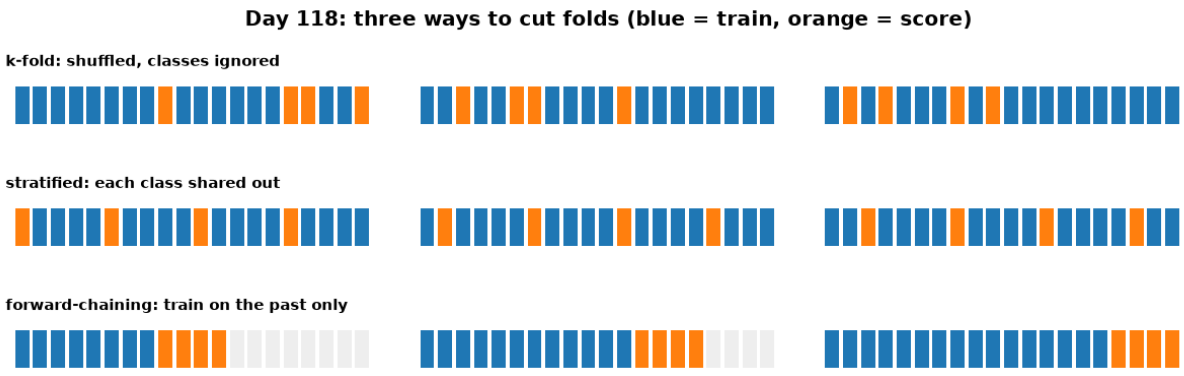
Three ways to cut folds

CODE

```
def kfold(n, k, seed):
    return np.array_split(np.random.RandomState(seed).permutation(n), k)
def stratified(y, k, seed):
    rng = np.random.RandomState(seed); folds = [[] for _ in range(k)]
    for c in np.unique(y):
        idx = rng.permutation(np.where(y == c)[0])
        for i, j in enumerate(idx):
            folds[i % k].append(j)
    return [np.array(sorted(f)) for f in folds]
def forward(n, k, min_train):
    edges = np.linspace(min_train, n, k + 1).astype(int)
    return [(np.arange(edges[i]), np.arange(edges[i], edges[i + 1])) for i in range(k)]
```

- `np.array_split(perm, k)` — k nearly equal pieces, even when n is not a multiple of k.
- `folds[i % k].append(j)` — % is the remainder: row 0 to fold 0, row 1 to fold 1, ..., row k back to fold 0, like dealing cards.
- `np.linspace(min_train, n, k + 1).astype(int)` — k + 1 evenly spaced edges: block i runs from edge i to edge i + 1.
- `(np.arange(edges[i]), ...)` — each forward fold trains on every row before its block: the past only.

SYSTEM DIAGRAM



Three ways to cut folds.

Segment 3 — Models and scores

Models from the parts

CODE

```
def svm_rbf(X, y, gamma, C):
    A, mu, sd = (X - X.mean(0)) / (X.std(0) + 1e-12), X.mean(0), X.std(0) + 1e-12; t = np.where(y
        == 1, 1.0, -1.0)
    K = rbf(A, A, gamma); a, _ = smo(K, t, C); b = bias(a, t, K, C)
    return lambda Z: decision(a, t, b, rbf(A, (Z - mu) / sd, gamma))
def knn(X, y, k):
    mu, sd = X.mean(0), X.std(0) + 1e-12
    return lambda Z: knn_predict((X - mu) / sd, y, (Z - mu) / sd, k)
def candidates():
    c = {f"SVM gamma {g} C {C}": (lambda X, y, g=g, C=C: (lambda f: lambda Z: (f(Z) >
        0).astype(int))(svm_rbf(X, y, g, C)))
         for g in (0.003, 0.03, 0.3) for C in (0.1, 1, 10)}
    c.update({f"kNN k {k}": (lambda X, y, k=k: knn(X, y, k)) for k in (1, 5, 15, 35)})
    return c
```

- `lambda Z: decision(a, t, b, rbf(A, (Z - mu) / sd, gamma))` — the fitted SVM as a function: new rows are scaled with the training mean and spread.
- `(lambda f: lambda Z: (f(Z) > 0).astype(int))(svm_rbf(...))` — a function made and called at once: it turns the SVM's scores into labels 0 and 1.
- `g=g, C=C` — default arguments freeze this pass's values; without them every lambda would use the last gamma and C.
- `c.update({...})` — adds the four kNNs to the same dictionary of candidates.

cv_score

MATH

$$CV = (1/k)\ \Sigma_i\ accuracy(\text{model trained without fold } i, \text{on fold } i)$$

CODE

```
def cv_score(fit, X, y, folds):
    out = []
    for i in range(len(folds)):
        va = folds[i]; tr = np.concatenate([folds[j] for j in range(len(folds)) if j != i])
        out.append(np.mean(fit(X[tr], y[tr])(X[va]) == y[va]))
    return float(np.mean(out))
```

- `np.concatenate([folds[j] for j in range(len(folds)) if j != i])` — every fold but the i-th, joined: the training rows for this round.
- `fit(X[tr], y[tr])(X[va])` — fit returns a predictor; the second pair of brackets calls it on the validation rows.

Segment 4 — Time series

Time series: lags, kNN regression, least squares

MATH

$x_t = (s_t, ..., s_{t+7}) \to \text{target } s_{t+8}$; $RMSE = \sqrt{(\text{mean } (\text{prediction} - \text{truth})^2)}$

CODE

```
def lagged(series, lags):
    X = np.stack([series[i:len(series) - lags + i] for i in range(lags)], 1)
    return X, series[lags:]
def knn_reg(X, y, k):
    return lambda Z: y[np.argpartition(sqdist(Z, X), k - 1, axis=1)[:, :k]].mean(1)
def linear_reg(X, y):
    A = np.column_stack([np.ones(len(X)), X]); w = np.linalg.lstsq(A, y, rcond=None)[0]
    return lambda Z: np.column_stack([np.ones(len(Z)), Z]) @ w
def rmse(p, y):
    return float(np.sqrt(np.mean((p - y) ** 2)))
```

- `np.stack([series[i:len(series) - lags + i] for i in range(lags)], 1)` — each column is the series shifted by one more step: row t holds values t to t + lags - 1.
- `np.argpartition(D, k - 1, axis=1)[:, :k]` — the k smallest distances in each row, without sorting the whole row.
- `y[...].mean(1)` — kNN regression: the average of the neighbours' next values.
- `np.linalg.lstsq(A, y, rcond=None)[0]` — least squares; the column of ones gives the intercept.

series: the two real records

CODE

```
def series():
    co2 = sm.datasets.co2.load_pandas().data["co2"].interpolate().to_numpy()
    sun = sm.datasets.sunspots.load_pandas().data["SUNACTIVITY"].to_numpy()
    return {"CO2 (weekly, ppm)": co2, "Sunspots (yearly)": sun}
```

- `sm.datasets.co2.load_pandas().data["co2"]` — a pandas column from statsmodels' bundled dataset.
- `.interpolate().to_numpy()` — fills the 59 missing weeks with straight lines, then converts to a NumPy array.

Segment 5 — The experiments

demo_checks: verbatim, by reading the exercise files

CODE

```
me = open(os.path.abspath(__file__), encoding="utf-8").read(); found, total, dup = 0, 0, []
for d in range(112, 118):
    src = open(os.path.join(HERE, f"day{d}_exercise.py"), encoding="utf-8").read(); lines =
        src.split("\n")
    for node in ast.parse(src).body:
        if isinstance(node, (ast.FunctionDef, ast.ClassDef)) and node.name not in HELPERS:
            text = "\n".join(lines[node.lineno - 1:node.end_lineno])
            if (d, node.name) in SKIP:
                dup.append(f"{node.name} (day{d})"); continue
            total += 1; found += text in me
```

- `isinstance(node, (ast.FunctionDef, ast.ClassDef))` — functions and classes only; imports and constants are skipped.

- `lines[node.lineno - 1:node.end_lineno]` — line numbers start at 1 and lists at 0, hence the - 1.
- `found += text in me` — True counts as 1.

demo_nested: the choice repeated inside each outer fold

CODE

```
cv = {n: cv_score(f, Xt, yt, stratified(yt, 5, s)) for n, f in C.items()}; best = max(cv,
    key=cv.get)
test = float(np.mean(C[best](Xt, yt)(X[te]) == y[te]))
outer = stratified(yt, 5, 100 + s); nested = []
for i in range(5):
    va = outer[i]; ft = np.concatenate([outer[j] for j in range(5) if j != i])
    inner = {n: cv_score(f, Xt[ft], yt[ft], stratified(yt[ft], 5, s)) for n, f in C.items()}
    nested.append(np.mean(C[max(inner, key=inner.get)](Xt[ft], yt[ft])(Xt[va]) == yt[va]))
rows.append((cv[best], float(np.mean(nested)), test, best))
```

- `max(cv, key=cv.get)` — the dictionary key with the largest value: the winning candidate's name.
- `inner = {...} on Xt[ft]` — the inner CV sees only the outer fold's training rows; the outer validation rows stay unseen.
- `rows.append((cv[best], nested, test, best))` — a tuple per split, unpacked later into an array of the three numbers.

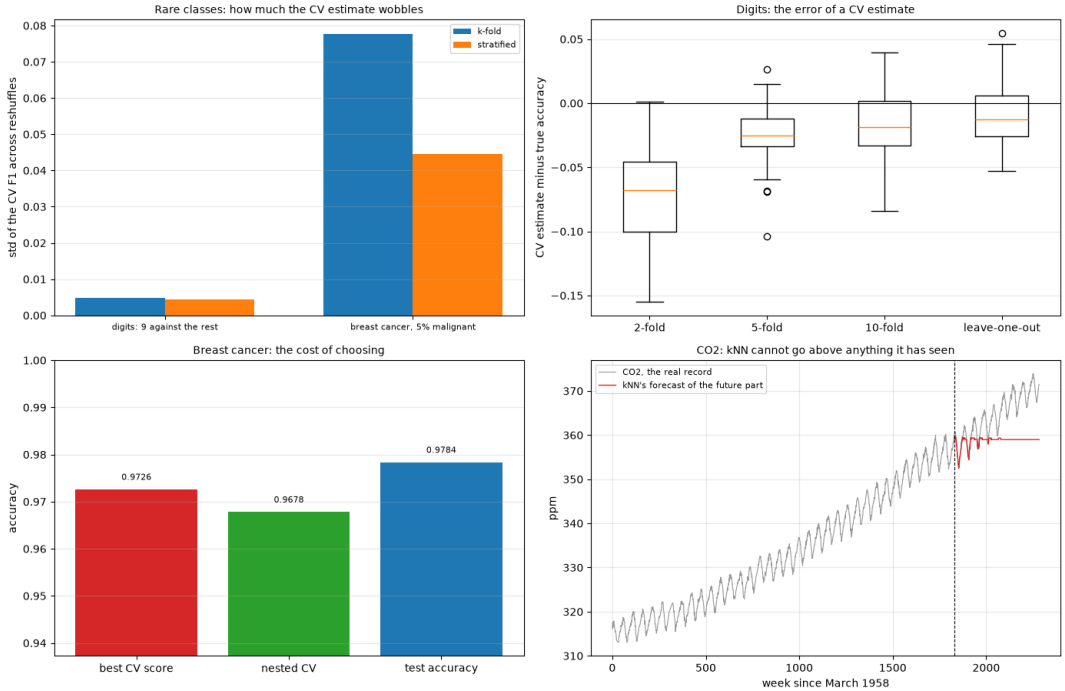
demo_time: shuffled against forward, and the real future

CODE

```
X, y = lagged(s, LAGS); cut = int(0.8 * len(y)); Xa, ya, Xf, yf = X[:cut], y[:cut], X[cut:],
    y[cut:]
print(f"\n{name}: {len(s)} values; past {cut} rows, future {len(y) - cut} rows; RMSE in the
    series' own units")
print(f"  {'model':<18}{'shuffled 5-fold':>17}{'forward-chaining':>18}{'the real future':>17}")
res = {}
for mname, make in (("kNN, k = 5", lambda A, b: knn_reg(A, b, 5)), ("linear on lags",
    linear_reg)):
    sh = []
    for f in kfold(len(ya), 5, 0):
        tr = np.setdiff1d(np.arange(len(ya)), f); sh.append(rmse(make(Xa[tr], ya[tr])(Xa[f]),
            ya[f]))
    fw = [rmse(make(Xa[tr], ya[tr])(Xa[te]), ya[te]) for tr, te in forward(len(ya), 5, len(ya) //
        3)]
    fut = rmse(make(Xa, ya)(Xf), yf); res[mname] = (np.mean(sh), np.mean(fw), fut)
```

- `X[:cut], X[cut:]` — the first 80% for choosing and estimating, the last 20% as the future: never shuffled.
- `np.setdiff1d(np.arange(len(ya)), f)` — every row index not in fold f.
- `forward(len(ya), 5, len(ya) // 3)` — the first third is always training; the rest is cut into five blocks.

REAL GRAPH



Real graph from the script: rare classes, CV error, the cost of choosing, and the CO2 forecast.

Quick Syntax Glossary — new or reinforced constructs

- `np.array_split(a, k)` — k nearly equal pieces.
- `i % k` — the remainder: deals rows round k folds.
- `max(d, key=d.get)` — the key with the largest value.
- `(lambda f: ...)(g)` — make a function and call it at once.
- `np.argpartition(D, k - 1, axis=1)` — the k smallest per row, unsorted.
- `np.setdiff1d(a, b)` — values in a that are not in b.

End of Day 118 syntax reference. For the experiments and what they showed, see the Day 118 complete guide.